

CACHING STRATEGIES

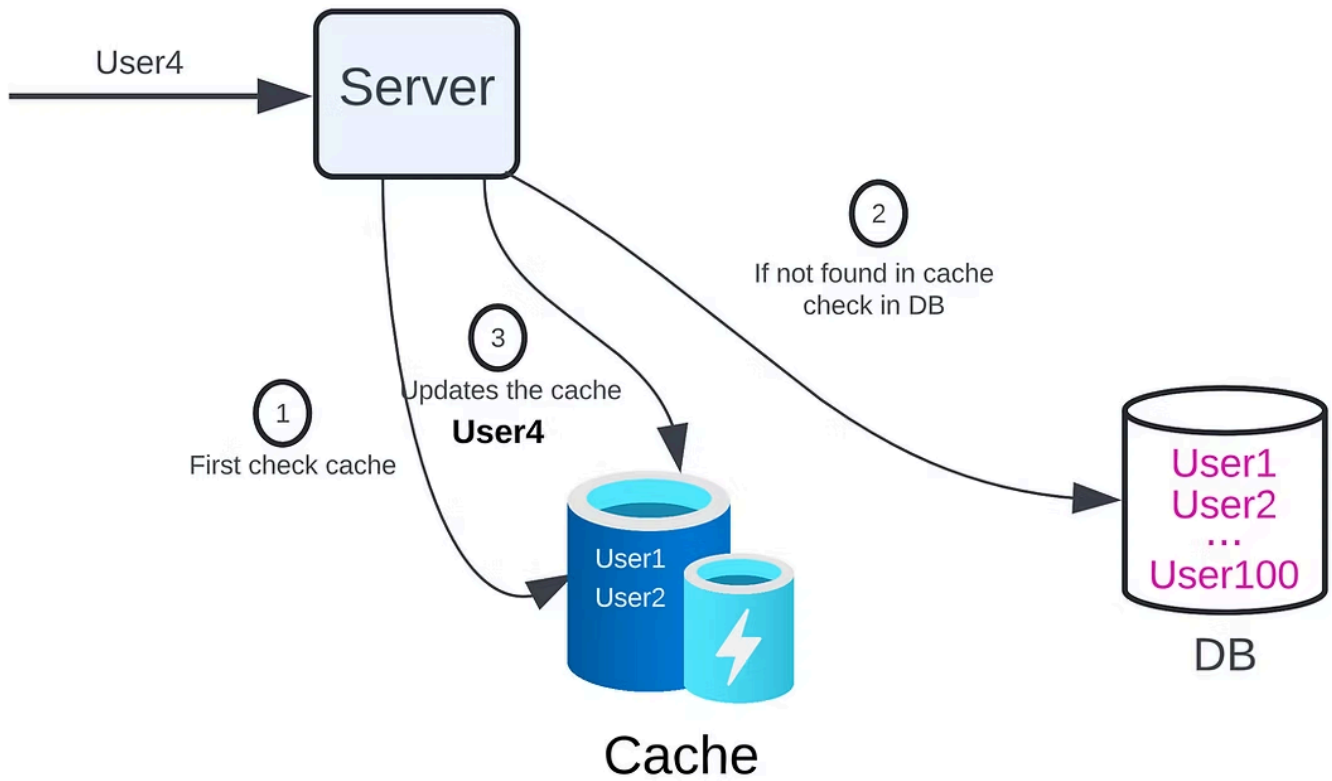
- 1.Caching Strategies:Read
 - 1.1.Cache Aside Strategy (CAS)
 - 1.1.1.CAS Pros
 - 1.1.2.CAS Cons
 - 1.2.Read Through Strategy (RTS)
 - 1.2.1.RTS Pros
 - 1.2.2.RTS Cons
- 2.Caching Strategies:Write
 - 2.1.Write Aside Strategy (WAS)
 - 2.1.1.WAS Pros
 - 2.1.2.WAS Cons
 - 2.2.Write Through Strategy (WTS)
 - 2.2.1.WTS Pros
 - 2.2.2.WTS Cons
 - 2.3.Write Behind Strategy (WBS)
 - 2.3.1.WBS Pros
 - 2.3.2.WBS Cons

Caching Strategies :Read

Here are two common methods to read data from a cache:

Cache Aside Strategy (CAS)

- When data is requested, we first look in the cache.
- If it's not there, we retrieve it from the database.
- The retrieved data is then saved in the cache for future requests.



CAS Pros:

- **Fault Tolerance:**

If the cache fails for any reason, the system doesn't break. We can still read data directly from the database, even though it may be a bit slower. Basically, the cache is kept separate (hence "cache-aside"), so it doesn't impact uptime if it goes down.

- **Flexible Schemas:**

Since the application controls the cache, we have more flexibility in how we store data. The cache and database can have different data structures. For example, the cache might only store commonly accessed details, like a user's name and contact details, while the database holds the full profile.

CAS Cons:

- **Stale Data:**

Let's say a user record (e.g., **user4**) is initially only in the database, not in the cache. When a read request comes in, the cache misses, so we fetch **user4** from the database and save it to the cache. Now, future requests for **user4** will be served from the cache.

But if **user4**'s record in the database is updated (say their address changes), the cache still holds the old address since it wasn't refreshed. If we get another request for **user4**, it will still serve the outdated address from the cache.

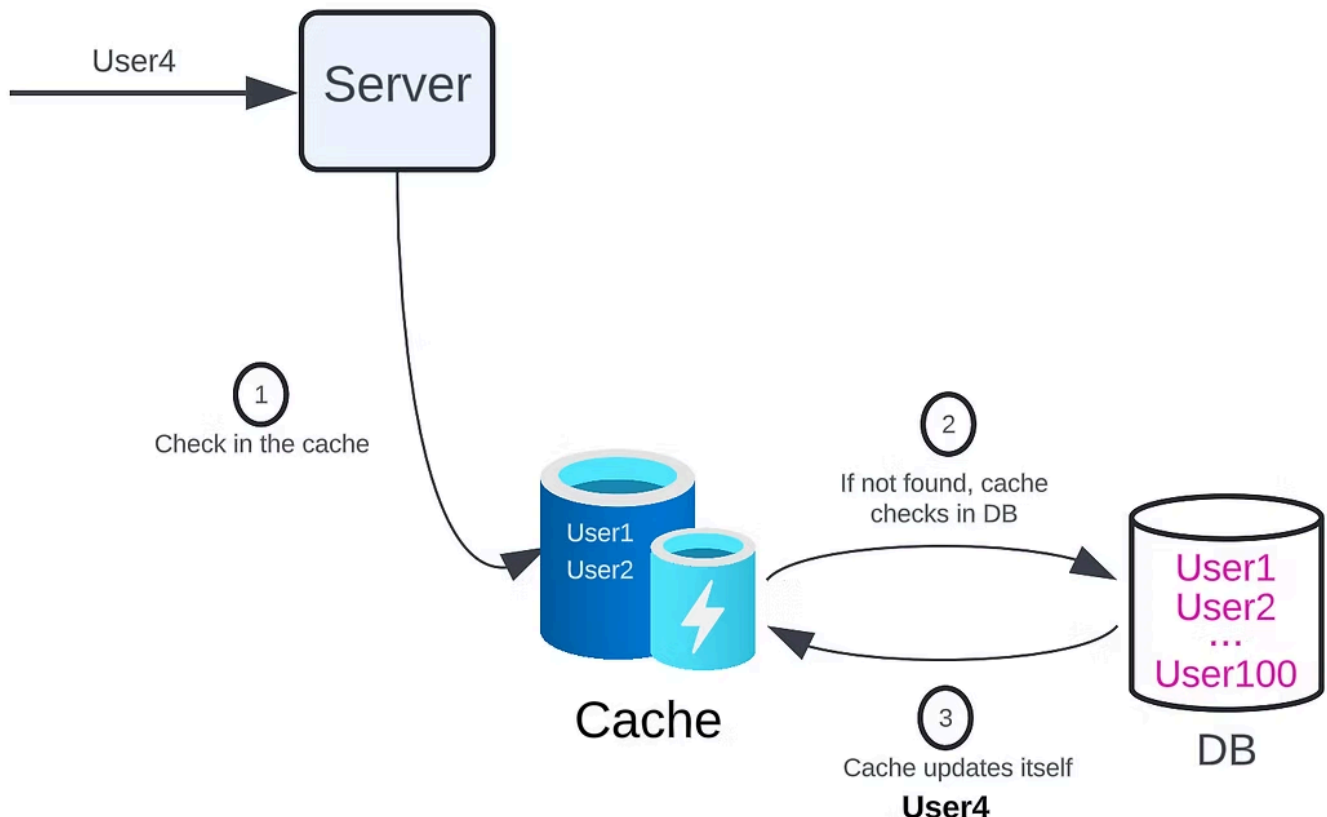
To avoid this, we need to periodically remove old data from the cache—a process called **cache invalidation**.

After invalidation, the next request for **user4** will miss the cache and fetch the updated info from the database.

Read Through Strategy (RTS)

- When data is requested, we first look in the cache.
- If it's not there, the cache itself fetches the data from the database.
- The cache then saves the data for future requests.

This is called "**Read Through**" because we are reading the data **through the cache**.



RTS Pros:

- **No Stale Data (with Write-Through):**

If we pair the Read-Through strategy with [Write-Through](#), data in the cache stays fresh. Every update (like a change in **user4**'s address) is written to both the cache and database at the same time. So, whenever we read data from the cache, it's guaranteed to be up-to-date.

RTS Cons:

- **Cache Failure Causes Downtime:**

If the cache fails, the application can't read data directly from the database because, in this strategy, all reads go through the cache. So, a cache failure could make the whole system unavailable.

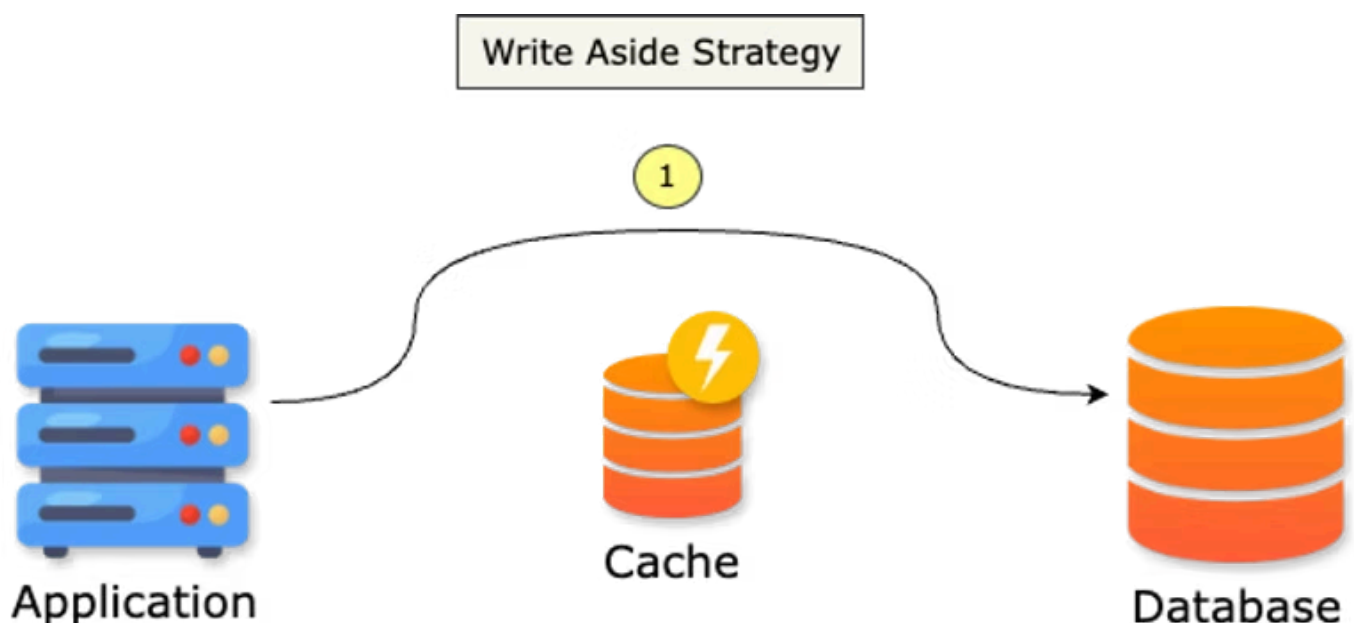
- **Non-Flexible Schema:**

In this approach, the cache and database need to have the same data structure. This limits flexibility since we can't customize what's stored in the cache.

Caching Strategies :Write

Here are the three common strategies to write the data in the cache:

Write Aside Strategy (WAS)



In this write strategy, data is written directly to the database, bypassing the cache.

You might wonder, “How does the cache get updated?” This happens during the read process! When the data is read, the system first checks the cache. If there’s a cache miss (meaning the data isn’t there), it fetches the data from the database and then updates the cache.

Basically, the cache gets updated reactively (on-demand) rather than proactively. This is why this approach is also known as **lazy caching**.

WAS Pros:

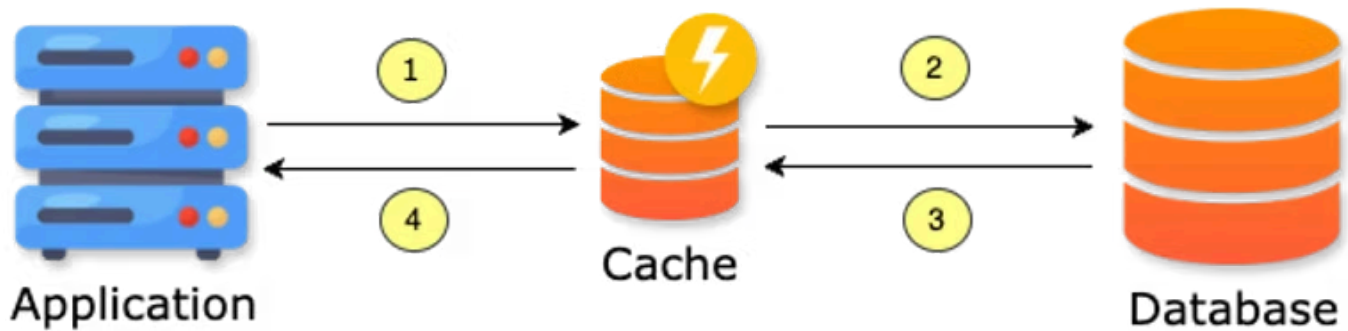
- **Faster Writes:** Data is written directly to the database, skipping the cache. Since we only write once (to the database), writes are faster, making it ideal for write-heavy systems where immediate cache updates aren't required.
- **Cache Independence:** The cache operates separately from the write process. If the cache fails, it doesn't impact database writes, ensuring system reliability.

WAS Cons:

- **Slower Reads:** Since the cache is bypassed during writes, data won't be in the cache immediately. Reads may result in a cache miss, requiring a fetch from the database before updating the cache, slowing down the process.
- **Risk of Stale Data:** If a user's address is updated in the database but the cache isn't refreshed, it may still contain the old data. Until the cache expires and removes stale data, users may see outdated information.

Write Through Strategy (WTS)

Write Through Strategy



In this strategy, every time data is written, it's first written to the cache. Then, the cache immediately updates the database.

This process happens **synchronously**—meaning that only after the database is updated does the application get confirmation of the write.

WTS Pros:

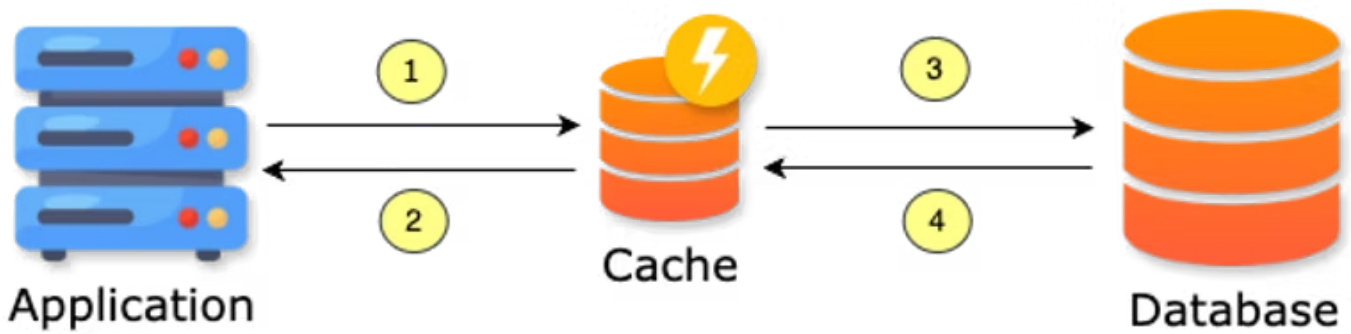
- **No Stale Data:** Every write updates both the cache and the database simultaneously. This ensures that updates, like a user's address change, are immediately reflected in both locations.
- **Faster Reads:** Since data is always available in the cache, reads are quick and do not rely on cache misses to update.

WTS Cons:

- **Slower Writes:** Writes take longer since they update both the cache and the database. Unlike the Write-Around strategy, this approach introduces extra latency.
- **Cache Dependency:** Since all writes go through the cache, a cache failure can prevent writes from reaching the database, potentially causing system downtime.

Write Behind Strategy (WBS)

Write Behind Strategy



In this strategy, when data is written, it's also first written to the cache, and the cache then updates the database.

The key difference here is that the database update happens **asynchronously**. The application receives a confirmation as soon as the data is written to the cache, and the database is updated in the background / asynchronously.

WBS Pros:

- **Faster Writes:** Data is first written to the cache, and the application receives a confirmation instantly. The cache then updates the database asynchronously in the background, improving write performance.
- **Faster Reads:** Since data is stored in the cache, read operations are quick and don't need to access the database.

WBS Cons:

- **Risk of Data Loss:** The asynchronous database update can fail. If the background update doesn't go through, data won't be stored in the database, raising durability concerns.
 - **Cache Dependency:** Since every write depends on the cache, a cache failure could prevent writes from being stored in the database, leading to system downtime.
-